

Evaluasi Kinerja Komputasi Paralel: Implementasi Parallel Merge Sort menggunakan OpenMPI

Fairuz Apuilla Rahagi

NIM: 13222107

Sekolah Teknik Elektro dan Informatika (STEI)

Institut Teknologi Bandung

29 April 2026

1 Pendahuluan

Laporan ini menyajikan hasil implementasi dan evaluasi algoritme pengurutan *Parallel Merge Sort* dalam lingkungan komputasi terdistribusi menggunakan pustaka *Message Passing Interface* (OpenMPI) menggunakan bahasa C++. Evaluasi dilakukan untuk mengukur efisiensi kinerja algoritme paralel dibandingkan dengan eksekusi sekuensial pada berbagai ukuran set data masukan.

2 Metodologi Eksekusi

Program dirancang dengan arsitektur *Root-Worker* dengan alur komputasi sebagai berikut:

1. **Distribusi (Scatter):** Proses *Root* (Rank 0) membaca berkas set data (berisi rentang 10^4 hingga 10^6 elemen *floating-point* acak) dan mendistribusikan potongan *array* secara merata kepada seluruh prosesor menggunakan fungsi `MPI_Scatterv`.
2. **Komputasi Paralel:** Setiap prosesor secara independen mengurutkan porsi datanya masing-masing menggunakan algoritme *Introsort* (`std::sort`).
3. **Penggabungan (Gather & Merge):** Data yang telah terurut dikumpulkan kembali ke proses *Root* menggunakan `MPI_Gatherv`, disusul dengan fase penggabungan akhir secara sekuensial menggunakan `std::inplace_merge`.

3 Hasil Pengujian

Pengujian kinerja dilakukan menggunakan skenario eksekusi dengan 1, 2, 4, dan 8 prosesor pada tiga variasi ukuran set data. Tabel 1 menunjukkan waktu komputasi murni yang diukur menggunakan fungsi `MPI_Wtime()`.

Table 1: Perbandingan Waktu Eksekusi (dalam detik)

Ukuran Data	1 Prosesor	2 Prosesor	4 Prosesor	8 Prosesor
10.000 elemen	0.004724	0.003561	0.002025	0.003823
100.000 elemen	0.008345	0.005633	0.006830	0.005977
1.000.000 elemen	0.144879	0.058076	0.045853	0.039539

4 Analisis Kinerja

Berdasarkan metrik eksekusi pada Tabel 1, terdapat beberapa karakteristik komputasi paralel yang teramati:

4.1 *Communication Overhead* pada Beban Rendah

Pada set data berukuran kecil (10.000 dan 100.000 elemen), penambahan jumlah prosesor (misalnya dari 4 ke 8 prosesor) justru menunjukkan degradasi performa (peningkatan waktu eksekusi). Hal ini disebabkan oleh *communication overhead*, di mana waktu latensi yang dihabiskan untuk mengatur komunikasi antar-proses MPI (*scatter* dan *gather*) mendominasi dan melebihi waktu komputasi pengurutan itu sendiri.

4.2 Efek *Cache* dan Skalabilitas Optimal

Pada set data masif (1.000.000 elemen), skalabilitas algoritme paralel terbukti efektif. Terjadi percepatan superlinear (*superlinear speedup*) yang drastis dari 1 prosesor (0.144 detik) ke 2 prosesor (0.058 detik). Penurunan waktu eksekusi hingga ~ 2.49 kali lipat ini merupakan indikasi kuat dari fenomena efisiensi hierarki memori, di mana pemecahan *array* berskala besar menjadi blok-blok memori yang lebih kecil memungkinkannya untuk ditampung secara optimal di dalam *Cache L2/L3* prosesor.

4.3 Hukum Amdahl dan *Bottleneck* Sekuensial

Meskipun terjadi peningkatan performa secara keseluruhan pada set data raksasa, efisiensi cenderung melandai saat menggunakan 8 prosesor. Sesuai dengan batasan

Hukum Amdahl (*Amdahl's Law*), batas atas dari percepatan program ini tertahan oleh keberadaan kode sekuensial, yakni tahapan `std::inplace_merge` yang dieksekusi secara mandiri oleh proses *Root* pada fase akhir penggabungan *array*.

5 Kesimpulan

Implementasi *Parallel Merge Sort* berbasis OpenMPI berhasil mendistribusikan dan menyelesaikan komputasi secara valid. Efisiensi komputasi terdistribusi baru akan optimal jika rasio beban komputasi jauh melampaui latensi komunikasi antar-simpul. Untuk optimasi tingkat lanjut, implementasi penggabungan berstruktur hierarki pohon (*Tree-based Parallel Merge*) dapat direkomendasikan guna mereduksi *bottleneck* sekuensial di proses *Root*.